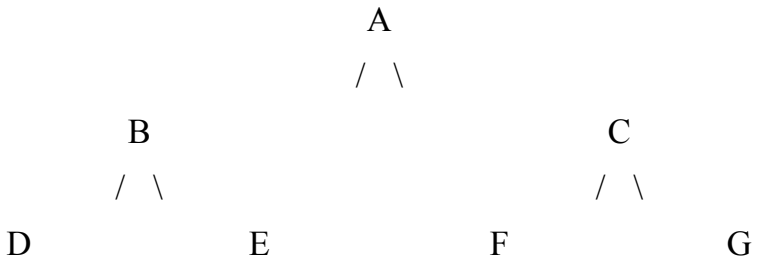
Question 1 – Breadth-First Search (BFS)

Step 1: Understand the problem

The graph is a tree with root **A**. The search starts at node **A**. BFS explores the graph level by level using a **Queue (First-In-First-Out)**. Children are added from left to right.



Levels: Level 0 = A; Level 1 = B, C; Level 2 = D, E, F, G.

Step 2: Algorithm rules applied

1. Put the start node A into the queue.
2. Remove (dequeue) the front node and mark it as **visited**.
3. Add all its unvisited children (left to right) at the rear of the queue.
4. Repeat steps 2 and 3 until the queue becomes empty.

Step 3: Dry run table

Queue is written front → rear.

Iteration	Queue (before dequeue)	Visited Node	Queue after expansion
1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G]
5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[ ] (empty)

Step 4: Answers to the questions

1. BFS traversal order:

A → B → C → D → E → F → G

2. Queue contents after each iteration:

Iteration 1: dequeue A → queue = [B, C]

Iteration 2: dequeue B → queue = [C, D, E]

Iteration 3: dequeue C → queue = [D, E, F, G]

Iteration 4: dequeue D → queue = [E, F, G]

Iteration 5: dequeue E → queue = [F, G]

Iteration 6: dequeue F → queue = [G]

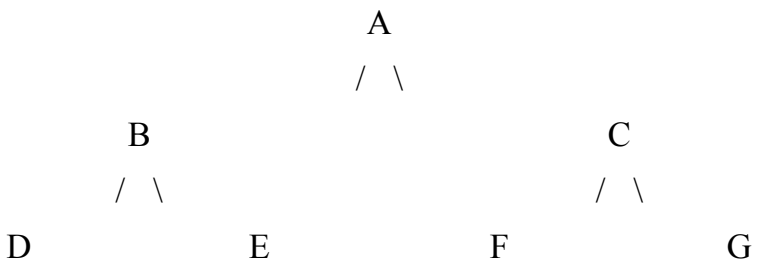
Iteration 7: dequeue G → queue = [ ] (empty)

Conclusion: All 7 nodes are visited in 7 iterations, level by level, and the queue is empty at the end. Because BFS uses a FIFO queue, every node of one level is visited before any node of the next level.

Question 2 – Depth-First Search (DFS)

Step 1: Understand the problem

Using the same graph, DFS starts from **A**. DFS goes as deep as possible along one branch before backtracking. It uses a **Stack (Last-In-First-Out)**. To visit the left child first, the children are pushed **right to left** (right child first, left child last, so the left child is on top).



Step 2: Algorithm rules applied

1. Push the start node A onto the stack.
2. Pop the top node and mark it as **visited**.
3. Push its unvisited children onto the stack (right child first, then left child).
4. Repeat steps 2 and 3 until the stack becomes empty.

Step 3: Dry run table

Stack is written bottom → top (the right-most item is the top).

Iteration	Stack (before pop)	Visited Node	Stack after push
1	[A]	A	[C, B]
2	[C, B]	B	[C, E, D]
3	[C, E, D]	D	[C, E]
4	[C, E]	E	[C]
5	[C]	C	[G, F]
6	[G, F]	F	[G]
7	[G]	G	[ ] (empty)

Step 4: Answers

DFS traversal order:

A → B → D → E → C → F → G

Explanation of the path:

1. Start at A and go down the left branch: A → B → D.
2. D is a leaf, so backtrack to B and visit its other child E.
3. E is a leaf and B has no more children, so backtrack to A and visit C.

4. From C go down the left branch to F, then backtrack and visit G.

Conclusion: All 7 nodes are visited in 7 iterations and the stack is empty at the end. DFS visits the nodes in the order A, B, D, E, C, F, G, whereas BFS visited them level by level (A, B, C, D, E, F, G).

Question 3 – BFS on the 8-Puzzle Problem

Step 1: Understand the problem

Initial State (S0)	Goal State
1 3 6 5 _ 2 4 7 8	1 2 3 4 5 6 7 8 _

(_ = blank tile, shown as 0 in the question)

Each state is a 3×3 arrangement of the tiles. A move slides the blank (0) **Up, Down, Left or Right**, and every move costs 1 (unweighted problem). BFS uses a **Queue (FIFO)**.

Step 2: Conventions used

1. Blank moves are generated in the order **Up, Down, Left, Right**.
2. Repeated states (already generated) are not added again.
3. The goal test is done when a state is removed from the queue.
4. States are named S0, S1, S2 … in the order they are generated.

Step 3: Dry run table (first 5 iterations)

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	1 3 6 5 _ 2 4 7 8 (S0)	[S0]	[S1, S2, S3, S4]
2	1 _ 6 5 3 2 4 7 8 (S1)	[S1, S2, S3, S4]	[S2, S3, S4, S5, S6]
3	1 3 6 5 7 2 4 _ 8 (S2)	[S2, S3, S4, S5, S6]	[S3, S4, S5, S6, S7, S8]
4	1 3 6 _ 5 2 4 7 8 (S3)	[S3, S4, S5, S6, S7, S8]	[S4, S5, S6, S7, S8, S9, S10]
5	1 3 6 5 2 _ 4 7 8 (S4)	[S4, S5, S6, S7, S8, S9, S10]	[S5, S6, S7, S8, S9, S10, S11, S12]

Meaning of the state names (children generated):

State	Obtained by	Layout	State	Obtained by	Layout
S1	S0 → Up	1 _ 6 5 3 2 4 7 8	S2	S0 → Down	1 3 6 5 7 2 4 _ 8
S3	S0 → Left	1 3 6 _ 5 2 4 7 8	S4	S0 → Right	1 3 6 5 2 _ 4 7 8
S5	S1 → Left	_ 1 6 5 3 2 4 7 8	S6	S1 → Right	1 6 _ 5 3 2 4 7 8
S7	S2 → Left	1 3 6 5 7 2 _ 4 8	S8	S2 → Right	1 3 6 5 7 2 4 8 _
S9	S3 → Up	_ 3 6 1 5 2 4 7 8	S10	S3 → Down	1 3 6 4 5 2 _ 7 8
S11	S4 → Up	1 3 _ 5 2 6 4 7 8	S12	S4 → Down	1 3 6 5 2 8 4 7 _

S0 is expanded first. Its parent is never regenerated: for example, S1 is not expanded back into S0. Iterations 6 to 311 continue in the same way (S5, S6, S7 … are expanded in the order of the queue) until the goal state is removed from the queue.

Step 4: Answers to the questions

1. Which node is expanded in each iteration?

Iteration	Node expanded	Iteration	Node expanded
1	S0 (initial)	4	S3
2	S1	5	S4
3	S2	6 – 311	S5 onwards, in queue order

Nodes are expanded in the order they entered the queue (FIFO). The goal state is the 311th state removed from the queue, so 310 states are expanded before it and the search stops at depth 8.

2. How many states are generated in total?

Following the conventions above, BFS generates **500 distinct states** in total (the initial state plus 499 new states) when the goal is reached. In the first five iterations, 12 new states (S1 to S12) are generated.

Note: if the goal test is done at the time a state is generated, the search stops earlier with 311 generated states. The solution path is the same.

3. Complete solution path (Initial State → Goal State):

Step	Move of blank	State
0	Start (S0)	1 3 6 5 _ 2 4 7 8
1	Right	1 3 6 5 2 _ 4 7 8
2	Up	1 3 _ 5 2 6 4 7 8
3	Left	1 _ 3 5 2 6 4 7 8
4	Down	1 2 3 5 _ 6 4 7 8
5	Left	1 2 3 _ 5 6 4 7 8
6	Down	1 2 3 4 5 6 _ 7 8
7	Right	1 2 3 4 5 6 7 _ 8
8	Right (Goal)	1 2 3 4 5 6 7 8 _

Move sequence: **Right → Up → Left → Down → Left → Down → Right → Right**. The solution has **8 moves** (path cost = 8).

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle?

1. All moves have the same cost (1), so the cost of a path equals its depth (number of moves).
2. BFS uses a FIFO queue, so all states at depth d are expanded before any state at depth d + 1.
3. Therefore the first time the goal is reached, it is at the smallest possible depth. Here, every state at depths 0 to 7 was examined and none was the goal, so no solution with fewer than 8 moves exists.
4. The branching factor is finite (at most 4), so BFS is complete and always finds a solution if one exists.

Conclusion: BFS reaches the goal state in 8 moves, which is the optimal (shortest) solution for this puzzle.